2012 IEEE/ACM International Conference on Advances in Social Networks Analysis and Mining

Using Pregel-like Large Scale Graph Processing Frameworks for Social Network Analysis

Louise Quick, Paul Wilkinson and David Hardcastle
Government Communications Headquarters
Hubble Road, Cheltenham, UK, GL51 0EX
Email: {louise.quick,paul.wilkinson,david.hardcastle}@gchq.gsi.gov.uk

***Abstract*—Pregel is a system for large scale graph processing developed at Google. It provides a scalable framework for running graph analytics on clusters of commodity machines. In this paper, we present several important undirected graph algorithms for social network analysis which fit within this framework. We discuss various graph componentisation methods, diameter estimation, degrees of separations, along with triangle, k-core and k-truss finding and computing clustering coefficients. Finally we present some experimental results using our own implementation of the Pregel framework, and examine key features of the general framework and algorithmic design.**

***Index Terms*—Distributed algorithms, Distributed computing, Graph theory, Parallel algorithms**

I. Introduction

In today's connected world there is an increasing need to analyse large scale graphs. For example, the World Wide Web hyperlink graph contains billions of nodes, Twitter has reported having over 100 million active users [1], and the active Facebook graph has 721 million nodes [2]. Understanding the structure and properties of graphs of this size and running algorithms over them is a significant computational challenge.

Pregel is a system for large scale graph processing developed by Google to tackle the problem of efficient processing in a parallel setting [3]. It is designed to be easy to use, scalable, flexible and fault-tolerant. As with MapReduce [4], the user does not have to think about the parallelism, only how to implement certain simple functions. Several initial algorithms were discussed in the original Pregel paper but there are no other references in the literature on the implementation of other algorithms in this framework.

In this paper, we discuss the implementation of several graph algorithms and metrics within this model of computation that are fundamental to giving an overall view of what a large social network looks like: componentisation, degrees of separation, diameter, k-cores, triangle finding, clustering coefficients and k-trusses. With the exception of degrees of separation and the diameter, all of the methods we present are exact. Further, we discuss key aspects of the framework and illustrate important and desirable features that any implementation of the Pregel framework should provide in order to allow flexible algorithm development.

A. Background and Related Work

Since its introduction, there has been an explosion in the number of MapReduce analytics written for the Hadoop [5] framework. Hadoop's ease of use, scalability and fault tolerant properties make it an attractive platform to run analytics.

Despite the fact that MapReduce is not a natural framework for expressing graph algorithms, many have been written in it (e.g. [6], [7]). However, they perform inefficiently due to their iterative nature, which causes the same data to be read from and written to disk multiple times.

Lin and Schatz [8] describe ways to improve the efficiency of these algorithms by reducing the amount of information needed to be shuffled across the cluster. Other work relating to graph and iterative algorithms in Hadoop include PEGASUS ([9], [10]), an open-source graph mining library which is implemented on top of Hadoop, and HaLoop [11], which provides a modified version of the Hadoop framework designed for iterative algorithms.

Three different platform models for processing large graphs (SQL Server PDW, DryadLINQ and Scalable Hyperlink Store) are evaluated in [12], which compare them using five basic graph algorithms. Other parallel graph frameworks include the Parallel BGL [13], CGMgraph [14], DisNet [15] and HIPG [16]. Current open-source implementations of the Pregel framework under active development are Giraph [17], GoldenOrb [18] and Spark/Bagel [19].

A recent study of the Facebook graph ([20], [2]) includes computation of path lengths, diameter estimation, clustering coefficients and connected components on graphs with 721 million nodes. Their analysis was performed on a large Hadoop cluster, using Hadoop/Hive [21], and HyperANF [22]. Other large scale studies which used some of the measures described here include MSN messenger [23] and Twitter [24].

B. Brief Details of the Framework

We provide a brief overview of the Pregel framework and refer the reader to [3] for further details. The system gives a simple model of computation with the parallelisation hidden behind the API. It was inspired by the Bulk Synchronous Parallel model, which guarantees synchronism between supersteps [25].

Graph algorithms in Pregel are thought of as a sequence of iterations, called *supersteps*. The algorithms are written from a node's point of view. In each superstep, every node in the graph can receive messages sent in the previous superstep, modify its state, mutate the graph topology and send messages

978-0-7695-4799-2/12 $26.00 © 2012 IEEE
DOI 10.1109/ASONAM.2012.254

to other nodes. The nodes, edges and messages can have arbitrary state.

The graph structure is stored in distributed RAM. Edges in the graph are stored as directed edges to allow each node to contain a list of its out-edges. The nodes, along with their out-edges, are partitioned across a cluster of machines. Nodes cannot however see their in-edges. Undirected edges must be modelled by two complimentary directed edges to allow all nodes to see the correct graph structure, or algorithms must be redesigned to take the directional aspect into account.

Nodes can be inactive during a superstep by voting to halt in the previous step. They are reactivated if they receive a message. The algorithms terminate when all nodes are inactive. During a superstep, only active nodes are processed.

Aggregators are a way of sharing global information about a computation. Each node can send a value to an aggregator in superstep S. These values are aggregated and redistributed to be read in superstep $S + 1$.

II. Componentisation

A fundamental problem in graph and social network analysis is to find the components of a graph. We find the connected components in an undirected graph by giving each node a label (this is their state). This label is initialised to the name of the node, updated throughout the algorithm based on messages the node receives (which contain other labels), and at termination contains the component the node belongs to. There are several ways, discussed below, in which this label information can be propagated and updated across the graph.

A. Parallel Componentisation

Similar to propagating a maximum value around a graph [3], at each superstep nodes receive messages containing potential labels. Initially, all nodes send their label to their neighbours. If any label received is lexicographically lower than the node's current label, the node changes its own label and passes the new label onto its neighbours. At each superstep nodes vote to halt and so when no more messages are passed around, the algorithm terminates (i.e. no node updated its state in the last superstep). This way, at termination, the components are labeled with the lowest lexicographically named node within their component. See Fig. 1 for pseudo-code. This contains an optimisation in the first superstep in which nodes scan over their neighbours to pick an initial label.

B. One-by-One Componentisation

The above method is very simple but does a lot of redundant work which requires a large amount of data transfer. A long chain of nodes in which the lowest lexicographically named node is at the end will produce many pointless intermediate messages which are passed along the chain.

In order to avoid this redundant work we describe a method which finds each component separately. We define three phases of this algorithm which are cycled through for each component in the graph.

```
1: Initialisation: Node label = node name
2: repeat
3:     begin Superstep n
4:         if Superstep 0 then
5:             Update label to be lowest of neighbours or keep own
6:         else
7:             Update label to be lowest from messages received or keep own
8:         end if
9:         if Updated label then
10:            Send label to neighbours
11:        end if
12:        Vote to halt
13:    end
14: until All nodes inactive
```

Fig. 1. Parallel Componentisation

Phases:

1) All nodes which have not yet found their component send their name to a *min-string* aggregator which selects the node to start at.
2) The node which has been selected via the aggregator sends messages to its neighbours containing its label.
3) This label is then propagated around the component, with nodes updating their labels and passing on the message. Nodes vote to halt once they have found their component.

Aggregators are an essential part of this method to decide which phase the computation is currently on and to determine when to move between phases. Whilst phase 1 and 2 only last one superstep each, the length of phase 3 depends on the size of the component being explored. Only after no more messages have been sent in phase 3 can we return to phase 1. This information again can be calculated through aggregators.

A major drawback to this method is having all nodes active (but doing no work) until their component is found. If however nodes voted to halt after phase 2 then nodes outside of the first component would never be reactivated. To avoid this we use an additional feature which allows a node to request the reactivation of all nodes within the graph. The master checks if any node has requested this reactivation of all nodes, and if one has, instructs the workers to reactivate all of their nodes.

We therefore require a node to know when to request this reactivation. The last node to update its label within a component will not know that it is the last and so we use the notion of a *lead* node which oversees the work in finding the component. It makes sense for this to be the starting node of each component (lowest named node) and so this node must always remain active, check when no more messages are passed around, and then request the reactivation, after which the computation returns to phase 1. See Fig. 2 for pseudo-code.

This feature is not discussed in the original paper but allows

```
1: Initialisation: Node label = node name
2: repeat
3:     begin Phase 1
4:         Nodes which haven't found their component send
                                their names to a min-string aggregator
5:     end
6:     begin Phase 2
7:         if Lowest named node then
8:             Send label to neighbours
9:             Become lead node
10:        else
11:            Vote to halt
12:        end if
13:    end
14:    begin Phase 3
15:        repeat
16:            If not already received the label, update own
                                label to be label in message
17:            if Updated label then
18:                Send label to neighbours
19:            end if
20:            Vote to halt (except lead node)
21:        until No messages sent around
22:        Lead node request reactivation of all nodes
23:    end
24: until All nodes found component
```

Fig. 2. One-by-One Componentisation.

less work to be done within supersteps as nodes can be inactive. We have found it to be a useful addition for this particular algorithm and it gives an extra layer of flexibility within the framework for other algorithms.

C. Combining Methods

Deciding which method to use depends on the type of graph that is being analysed. A graph with many small components lends itself to the parallel method since there are fewer redundant messages being sent around. Applying the one-by-one method to a graph with many small components, however, would not be efficient since to find each component several supersteps are used and aggregating the lowest named node is required for each component. On the other hand, a graph with a few large components would benefit from the one-by-one method.

Other graphs may have one giant component along with many smaller ones and in this case it is unclear which method is best. This leads us to introduce a method which combines the two approaches.

The chances are, the giant component would be found fairly early on in the one-by-one method and after this, with only small components remaining, reverting to the parallel method would speed up the outstanding processing. We call this the *mop-up* stage.

In order to decide when to revert to the mop-up stage we use a *mop-up threshold*, a point at which after this percentage of nodes have found their components, the computation reverts to the parallel algorithm.

D. Extensions

An alternative approach uses multiple starting nodes, which can increase the chances of hitting the giant component first. If two or more happen to be in the same component then an aggregator can be used to keep track of the merging components. Nodes which happen to receive two or more labels select the first label they receive and pass it on, and ignore the others whilst aggregating the merge information. In a final step, nodes need to check the aggregated merge information to determine final component assignment.

We discuss and compare the performance of all of the componentisation algorithms in section VII.

III. Diameter Estimation and Average Path Length

Single source shortest path finding is one of the simplest algorithms to implement in a Pregel framework. Discussed originally in [3], messages containing potential shortest distances are propagated out from a source node. Nodes receiving messages update their state to store their current shortest distance to the source node, and pass the message on to their neighbours.

This can easily be extended to use multiple source nodes. Each node simply contains a list of the source nodes and their current distances to each of them respectively. The messages must contain which source node the distance is relating to.

The diameter of a graph can be estimated using the *double sweep fringe* algorithm [26]. This only uses breadth first searches, from single and multiple starting nodes. It finds upper and lower bounds for the diameter and is iterated over until hopefully the bounds match.

Implementing this in Pregel frameworks requires extensive use of the aggregators, which are used to define which phase of the algorithm we are on, the current upper and lower bounds, the number of iterations, and to select new random starting nodes at each iteration. Reactivate all nodes was also used here to restart each BFS.

The degrees of separation in social network analysis is a well-known phenomena. Finding the average degree of separation between two nodes requires the distribution of path lengths within the graph which can again be found through BFSs. For large graphs, finding all pairwise distances is computationally hard and so instead a sample can be taken by running a multiple source shortest path algorithm. The more starting nodes, the more accurate the estimation will be. The number of starting nodes that can be computed depends on the size of the graph. A small graph will allow you to start from all nodes whilst the larger the graph is, the fewer starting nodes you can have.

IV. K-Core Finding

A k-core is a graph in which every node has degree of at least k. Finding the k-cores of an undirected graph can be done by iteratively removing nodes of degree less than k. We

```
1: repeat
2:     begin Superstep n
3:         if Node degree < k  then
4:             Delete node and out-edges
5:             Delete in-edges
6:         end if
7:     end
8: until No nodes deleted in previous superstep
9: All remaining nodes vote to halt
```

Fig. 3. K-Core Method 1.

```
1: repeat
2:     begin Superstep n
3:         for Messages received
4:             Delete corresponding out-edge
5:         end for
6:         if New degree < k then
7:             Delete node and out-edges
8:             Send message to neighbours
9:         end if
10:        Vote to halt
11:    end
12: until All nodes inactive
```

Fig. 4. K-Core Method 2.

discuss two methods for implementing this algorithm in the Pregel framework. They both use the ability to mutate the topology of the graph throughout the computation.

A. Method 1

In each superstep, nodes count their degree and request to delete themselves if it is less than k. They must also request the deletion of any edges which point to them. This is known since we have an undirected graph (represented in the directional framework with edges stored in both directions) and so any out-edge has a corresponding in-edge.

In order to terminate the algorithm we aggregate the number of nodes removed at each superstep. When this is 0, all nodes can vote to halt and the computation stops. Fig. 3 gives pseudo-code.

B. Method 2

Method 1 requires each node to be active in all supersteps and compute its degree. For nodes within high degree cliques, each of these nodes are doing repeated work. In order for nodes to become inactive there needs to be a way for them to become active when they are required to re-compute their degree. We use message passing to do this. Instead of a node requesting to remotely delete all edges pointing to it, it sends a message to the other nodes telling them to delete that edge. This activates the nodes when they need to recalculate their degree. Since edge deletion occurs between supersteps, nodes must not count the 'to be deleted' edges when they re-compute their degree. Fig. 4 gives pseudo-code.

Some of the messages sent are in fact sent to nodes which have just been deleted, but this is handled by the framework. Method 1 requires no explicit message passing[1] but does require all nodes to be active. There is clearly a trade-off between reducing the number of active nodes and having no messages being sent around. We discuss the performance in section VII.

V. Triangle Finding and Clustering Coefficient

The global and local clustering coefficients are well studied measures which calculate how prevalent triangles are within a graph. They can be used to compare networks and nodes within graphs.

Calculating these quantities requires local and global knowledge about the number of triangles within the graph, a property that is also required when finding k-trusses.

To find the triangles within a graph, we use the technique of enumerating open triads and spotting when the triads are closed, an approach used in Cohen's MapReduce implementation [6]. Our implementation also picks up on ideas in Suri and Vassilvitskii [27] for reducing the amount of work that is needed to be done.

Each node can only see two sides of a potential triangle as it cannot see the edges belonging to its neighbours, and so in order to find any triangles, messages must be passed to question the nodes about the existence of the completing third side of the triangle.

In order to avoid doing three times the work, we use an ordering of the nodes to ensure only one node is responsible for finding each triangle. To balance the work amongst nodes so that high degree nodes do not end up doing a lot of work, we order the nodes based on their degrees, reverting to lexicographical ordering when they are identical, and allow the low degree nodes to do the initial work. This also enables us to save space by storing edges in only one direction (from the low degree node to the high degree node) since the reverse edge is not required for the computation. Augmenting the input file with edge degrees can be done with a simple MapReduce job [6].

In the first Superstep, each node scans over its out-edges and for each pair of edges it contains, it must send a message to one querying the existence of the completing third edge. In order to know which node to send the message to, the edges must store the degree of the destination node on it, since the completing third edge, if it exists, will be directed from the lowest degree node of the two, to the highest. Nodes can then vote to halt.

In the next Superstep, nodes which receive messages check their edges to see if one corresponding to the message exists. If it does then a triangle has been found, and if not it is just an open triad.

Nodes which have found triangles can then complete the computation depending on what is required. In order to

[1]There is implicit message passing; a node or edge deletion request is another form of message.

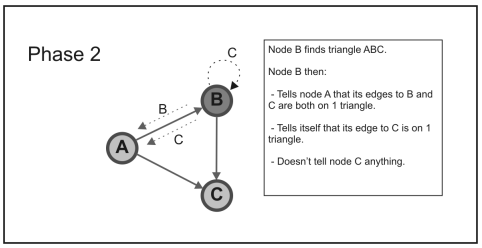

Fig. 5. K-Truss Phase 2 Work.

calculate the local clustering coefficient, each node requires the total number of triangles it is involved in, and so the nodes which have found the triangles can send one message to each of the nodes in the triangle. In the next Superstep, nodes simply count the number of messages they receive.

The final calculation for local clustering coefficient requires each node to know the number of neighbours they have. Since we have only stored the edges in one direction, this is not straight forward. It is therefore required that each node store the actual number of edges it has in their node state. The global clustering coefficient can easily be found using aggregators within the same computation as the local clustering coefficients.

VI. K-Truss Finding

A k-truss is a non-trivial, one-component subgraph such that each edge is reinforced by at least $k-2$ pairs of edges making a triangle with that edge [28]. It is a relaxation of a clique which can be efficiently computed and finds subgraphs based on the observation that strong ties are likely to be reinforced through other nodes. In Cohen's MapReduce implementation [6], an iterative process is used where edges are dropped if they don't have sufficient support (where support is defined as the number of triangles an edge participates in). As above, we store the edges in one direction only.

Fig. 6 gives pseudo-code for the algorithm. It is split into four phases, each lasting a superstep. The first two find the triangles as before. Once triangles have been found, messages are sent to each edge source in the triangle (see Fig. 5). Nodes collate their messages in phase 3 and delete edges with insufficient support. Nodes also need to know when to delete themselves, however, since we are only storing edges in one direction, nodes will not know if any edges are pointing to them. Therefore, for each edge with sufficient support, we send a blank message to the edge source and destination nodes. Nodes not receiving messages in phase 4 do not have any in or out edges and so can delete themselves. This process is iterated over until no nodes or edges are deleted.

One of the key differences between the MapReduce implementation and this Pregel implementation is MapReduce's ability to bin the triangle information by edges, giving a convenient way to count the support of an edge. The Pregel framework is node centric meaning the nodes have to count the support for each of their edges.

```
1:  repeat
2:      begin Phase 1
3:          Send messages about completing third edges
4:      end
5:      begin Phase 2
6:          for Messages received
7:              if Completing third edge exists then
8:                  Send message to edge source for each edge
                                                        in triangle
9:              end if
10:         end for
11:     end
12:     begin Phase 3
13:         Collate messages for each edge
14:         for Edges
15:             if Support < k − 2 then
16:                 Delete edge
17:             else
18:                 Send message to edge source and edge
                                                    destination nodes
19:             end if
20:         end for
21:     end
22:     begin Phase 4
23:         if No messages received then
24:             Delete node
25:         end if
26:     end
27: until No nodes or edges deleted in previous iteration
28: All remaining nodes vote to halt
```

Fig. 6. K-Truss Algorithm.

VII. Experimental Results

We report runtimes for the algorithms described above to compare our different methods, contrast with MapReduce, and provide an overall performance summary of this framework. We used our own internal implementation of the framework, based on top of Hadoop, because at the start of this research, no other open-source frameworks were available.

We used a cluster of 120 machines each configured to use 4GB of RAM for the process. We ran each experiment three times and took the average runtime. Our tests were run using a randomly generated graph with 10 million nodes, and a log-normal distribution of out-degrees (d),

$$p(d+1) = \frac{1}{\sqrt{2\pi}\sigma d} e^{-(\ln d - \mu)^2/2\sigma^2}$$

with $\mu = 2.5$ and $\sigma = 1$. We then made the edges undirected as the algorithms discussed in this paper are for undirected graphs. This gave a graph with a mean degree of 38, giving a total of approximately 380 million edges.

The Pregel framework is designed to be scalable and certainly 10 million nodes is not near any sort of limit on the size of graph that can be processed. Since the graph is stored in RAM there is a trade off between the size and density of

graph which can be evaluated. Similarly, a larger cluster would enable larger graphs to be analysed.

Where we compare different methods, different implementations of the framework may behave in varying ways depending on their design, and similarly for different graphs. It is, however, believed that the broad principles found during the experiments are applicable across the different implementations available.

The runtimes reported include the time taken to load the graph into RAM from HDFS and write out the results back to HDFS. The average load time was 23 seconds and whilst the write time varied depending on the amount written out, it was roughly 10 seconds.

A. Componentisation Results

We discussed several methods for componentisation. The results for our main test graph, along with three other random graphs are given in Table I. We gave the mixture methods a mop-up threshold of 50% and used an initial probability of 0.0000001 of being selected as a starting node in the multi-start method.

For the main graph, our best result came using the one-by-one method. As the graph only had one component it ran quickly, with much fewer messages being sent compared to the parallel method (378 million vs. 1.7 billion). The multi and intel starting methods were also very comparable.

The graph with 5 components performed very well with the intel-start method, however we could have just been lucky with our choice of start node. It also did well with the one-by-one method method.

A graph with 12 components performed much better with the parallel algorithm, and as the number of components increases, this difference will only get larger.

It is clear that the graph with 30 million components would not componentise well using the one-by-one method. The intel-start method gave the best time as the small components were mopped up quickly.

Where the boundaries lie between using parallel, one-by-one and the mixture methods is unclear, although componentising graphs nearing these boundaries with the wrong method would not cause too much of a performance hit.

A comparison with a MapReduce implementation saw the runtimes decrease by approximately 80%.

B. Other Results

Table II gives our other results for the main graph. We were able to estimate the degrees of separation using a sample of approximately 0.0001% of all possible pairwise paths. To get a better estimate, this could be run several times.

The k-core results show that having more Supersteps did not alter the time much. Supersteps were quick and completed in a matter of seconds. This highlights the benefit of the iterative algorithms over MapReduce.

There is little difference between the two k-core methods ultimately highlighting that the same sort of work is being done in both. We have seen examples where method two out performs method one and expect this to be due to properties of the graph.

We ran a MapReduce implementation of the k-core algorithm, which took 182s and 570s for 10 and 22 core respectively. The advantage where more iterations were required is clear.

We are able to compare our triangle results with Suri and Vassilvitskii [27]. They report a runtime of 5 and a half minutes for a MapReduce triangle finding algorithm using a cluster of size 1636 and a graph with 4.8 million edges and 86 million nodes. Our time is almost half of this, on a cluster less than a tenth of the size and a graph twice as big.

The runtimes for calculating the clustering coefficient and truss finding are dominated by the triangle finding aspect. Once this has been done, the remaining calculations are quick.

TABLE I
COMPONENTISATION RESULTS IN SECONDS FOR 1) PARALLEL, 2) ONE-BY-ONE, 3) MIXTURE, 4) MULTI-START AND 5) INTEL-START METHODS.

	Main Graph 1 Comp. $n = 10M$ $e = 378M$	Graph 2 5 Comps. of sizes 50,5,2,2,1M $n = 60M$ $e = 238M$	Graph 3 12 Comps. of size 5M each $n = 60M$ $e = 458M$	Graph 4 30M Comps. Various sizes incl. GC $n = 177M$ $e = 191M$
1	259	481	458	760
2	173	244	692	NA
3	NA	421	683	837
4	177	292	427	590
5	178	241	524	513

TABLE II
PERFORMANCE RESULTS.

Method	# Supersteps	Time(s)
Degrees of Separation	8	287
10-Core One	4	128
10-Core Two	5	141
22-Core One	19	161
22-Core Two	20	195
Enumerating Triangles	2	233
Clustering Coefficient	3	240
5-Truss	9	232

VIII. CONCLUSIONS AND FURTHER WORK

We have found it easy to express many algorithms within the Pregel framework. We have developed several fundamental graph algorithms which are key to understanding social networks and highlighted key features of the framework. We have discussed various different methods, compared their runtimes, and proved the framework is suitable for analysis of social networks at scale.

Having researched and tested many different algorithms, several key observations about algorithmic design and framework flexibility emerged which we list below.

1) Reducing the amount of work in supersteps, even if this increases the overall number of supersteps, pays off.
2) Reactivating all nodes is used in our componentisation methods showing performance gains for certain types

of graphs over a naïve method. Other algorithms also benefited from this feature (e.g. diameter estimation).

3) Having inactive nodes reduces the amount of work done. The framework has all nodes initially active which seems wasteful for certain algorithms (e.g. single source shortest path and some versions of componentisation). Having an option to specify which nodes are initially active (on input) would save time.
4) Aggregators are a key part of many algorithms. In particular the ability to use custom aggregators to define certain operations is useful for various algorithms, and would be used heavily in more complex algorithms.
5) Aggregators are either reset at each superstep, or they are *sticky* and persist through all supersteps. It would be useful to have *semi-sticky* aggregators where users can specify their time to live, or can indicate when they should be reset. They could be used in diameter estimation to keep track of the progress of a BFS.
6) The quantity of interest is not always the final graph, but sometimes the aggregator values. Implementations of the framework should provide an easy way in which to specify the type of output required.
7) We have seen that many ideas used in MapReduce formulations of the algorithms have been applicable in our implementations in the Pregel framework, making MapReduce a good starting point for further algorithm suggestions and design.
8) We have found the framework flexible for use with large graphs but also with smaller graphs with richer metadata.

Development of further algorithms and efficient methods for this framework is required in order to guide open-source software into flexible and useable systems.

The framework is also flexible enough to implement generic iterative algorithms. By thinking of each data point as a node, with no edges, k-means can be implemented. Each node computes its nearest cluster, with aggregators providing a mechanism for computing future cluster means. This can also be extended to the Gaussian Expectation-Maximisation algorithm. Building classifiers could also be implemented by have a set of training nodes and a set of test nodes, which are inactive whilst the other set is working.

REFERENCES

[1] Michael Busch, Krishna Gade, Brian Larson, Patrick Lok, Samuel Luckenbill, and Jimmy Lin, *Earlybird: Real-Time Search at Twitter* in Proc. 28th Intl. Conf. on Data Engineering (ICDE 2012), Washington, DC, April 2012.

[2] Johan Ugander, Brian Karrer, Lars Backstrom, and Cameron Marlow, *The Anatomy of the Facebook Social Graph*. http://arxiv.org/abs/111.4503, 2011.

[3] Grzegorz Malewicz, Matthew H. Austern, Aart J.C. Bik, James Dehnert, Ilan Horn, Naty Leiser, and Grzegorz Czajkowski, *Pregel: A System for Large-Scale Graph Processing*. in Proc. ACM SIGMOD Intl. Conf. Management of Data 2010, Indianapolis, IN, June 2010, 135-146.

[4] Jeffrey Dean and Sanjay Ghemawat, *MapReduce: Simplified Data Processing on Large Clusters*. in Proc. 6th Symp. Operating Systems Design and Implementation (OSDI 2004), San Fransisco, CA, Dec. 2004, 137-150.

[5] Andrzej Bialecki, Michael Cafarella, Doug Cutting, and Owen O'Malley. *Hadoop: A Framework for Running Applications on Large Clusters Built of Commodity Hardware*. http://lucene.apache.org/hadoop/, 2005.

[6] Jonathan D. Cohen, *Graph Twiddling in a MapReduce World*. Computing in Science and Engineering, July/August 2009, 29-41.

[7] Jimmy Lin and Chris Dyer. *Data-Intensive Text Processing with MapReduce*. Morgan & Claypool, 2010.

[8] Jimmy Lin and Michael Schatz, *Design Patterns for Efficient Graph Algorithms in MapReduce*. in Proc. 8th Workshop on Mining and Learning with Graphs (MLG 2010), Washington, DC, Aug 2010.

[9] U Kang, Charalampos E. Tsourakakis, and Christos Faloutsos, *PEGASUS: A Peta-Scale Graph Mining System*. in Proc. IEEE Intl. Conf. Data Mining (ICDM 2009), Miami, FL, Dec. 2009, 229-238.

[10] U Kang, Charalampos E. Tsourakakis, and Christos Faloutsos, *PEGASUS: Mining Peta-Scale Graphs*. Knowledge and Information Systems. 27(2), May 2011, 303-325.

[11] Yingyi Bu, Bill Hawe, Magdalena Balazinska, and Michael Ernst, *HaLoop: Efficient Iterative Data Processing on Large Clusters*. in Proc. 36th Intl. Conf. Very Large Data Bases (VLDB 2010), Singapore, 2010, 285-296.

[12] Marc Najork, Dennis fetterly, Alan Halverson, Krishnaram Kenthapadi, and Sreenivas Gollapudi, *Of Hammers and Nails: An Empirical Comparison of Three Paradigms for Processing Large Graphs*. in Proc. 5th Intl. Conf. on Web Search and Data Mining (WSDM 2012), Seattle, February 2012.

[13] Douglas Gregor and Andrew Lumsdaine, *The Parallel BGL: A Generic Library for Distributed Graph Computations*. in Proc. Parallel Object-Oriented Scientific Computing (POOSC), July 2005.

[14] Albert Chan and Frank Dehne, *CGMGRAPH/CGMLIB: Implementing and Testing CGM Graph Algorithms on PC Clusters and Shared Memory Machines*. Intl. J. of High Performance Computing Applications. 19(1),2005,81-97.

[15] Ryan Lichtenwalter and Nitest V. Chawla, *DisNet: A Framework for Distributed Graph computation* in Proc. ACM/IEEE Conference on Advances in Social Network Analysis and Mining (ASONAM 2011), Kaohsiung, Taiwan, 2011.

[16] Elzbieta Krepska, Thilo Kielmann, Wan Fokkink, and Henri Bal, *High-Level Framework for Distributed Processing of Large-Scale Graphs*. in Proc. 12th Intl. Conf. on Distributed Computing and Networking (ICDCN 2011), Bangalore, India, Jan 2011, 155-166.

[17] Giraph. *https://github.com/aching/Giraph*.

[18] Golden-Orb. *http://www.raveldata.com/goldenorb*.

[19] Bagel. *https://github.com/mesos/spark/pull/48*.

[20] Lars Backstrom, Paolo Bolidi, Marco Rosa, Johan Ugander, and Sebastiano Vigna, *Four Degrees of Separation*. http://arxiv.org/abs/111.4570, 2011.

[21] Ashisi Thusoo, Joydeep Sen Sarma, Namit Jain, Zheng Shao, Prasad Chakka, Ning Zhang, Suresh Antay, Hao Lin, and Ragthotham Marthy, *Hive - a Petabyte Scale Data Warehouse using Hadoop*. in Proc. 26th Intl. Conf. on Data Engineering (ICDE 2010), Long Beach, CA, March 2010.

[22] Paolo Boldi, Marco Rosa, and Sebastiano Vigna *HyperANF: Approximating the Neighbourhood Function of Very Large Graphs on a Budget*. in Proc. 20th Intl. Conf. on World Wide Web (WWW 2011), Hyderbad, India, March, 2011.

[23] Jure Leskovec and Eric Horvitz, *Planetary-Scale Views on a Large Instant-Messaging Network*. in Proc. 17th Intl. Conf. on World Wide Web (WWW 2008), Beijing, China, April 2008.

[24] Haewoon Kwak, Changhyun Lee, Hosung Pak, and Sue Moon, *What is Twitter, a Social Network or a News Media?*. in Proc. 19th Intl. Conf. on World Wide Web (WWW 2010), Raleigh, NC, April 2010.

[25] Leslie G. Valiant, *A bridging Model for Parallel Computation*. Comm. ACM 33(8), 1990, 103-11.

[26] Pierluigi Crescenzi, Roberto Grossi, Claudio Imbrenda, Leonardo Lanzi, and Andrea Marino, *Finding the Diameter in Real-World Graphs: Experimentally Turning a Lower Bound into an Upper Bound*. Lecture Notes in Computer Science. Part 1, v.6346,2010, 302-313.

[27] Siddharth Suri and Sergei Vassilvitskii, *Counting Triangles and the Curse of the Last Reducer*. in Proc. 20th Intl. Conf. World Wide Web (WWW 2011), Hyderabad, India, March 2011, 607-614.

[28] Jonathan D. Cohen. *Trusses: Cohesive Subgraphs for Social Network Analysis*. https://www.computer.org/cms/Computer.org/dl/mags/cs/2009/04/extras/msp2009040029s1.pdf.